HARNESS LAB · MÓDULO 3 · AULA 3.2

Guidelines *do seu projeto*

Tem ferramenta que escreve o CLAUDE.md por você. No Laravel, o Laravel Boost. Em qualquer outra stack, o boostify, com a mesma filosofia: a versão instalada, um bloco marcado e a documentação certa a uma consulta de distância.

`laravel/boost` `boost:install` `<laravel-boost-guidelines>` `.ai/guidelines` `/boostify:boostify` `context7` `claude-code-setup` `AGENTS.md` `paths:`

VOCÊ SAI COM

Um CLAUDE.md com o bloco gerado para as versões que o seu projeto usa (Boost ou boostify), o que só o seu time sabe fora do bloco, e o arquivo inteiro abaixo de 200 linhas.

Material de apoio · Lucas Souza · Beer and Code

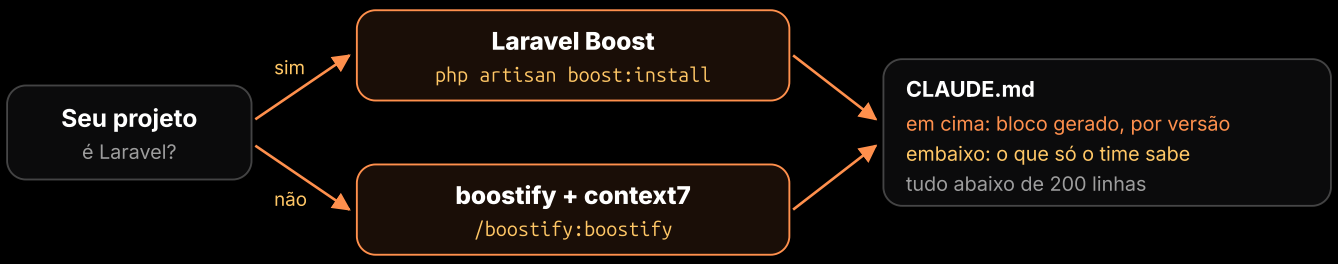
1 A aula em *uma página*

Guidelines são a construção do CLAUDE.md. A aula abre com um recado para quem é do Laravel: a única coisa a fazer é instalar o Laravel Boost e não ser teimoso. Nada de escrever os seus "erros comuns" por cima. O Boost tem as regras exatas, bem escritas e validadas pela comunidade sobre como montar um bom CLAUDE.md.

Na gravação, o projeto já tinha o Boost (aplicação nova), então bastou o `sail artisan boost:install`. Ele pergunta o que montar: **AI Guidelines**, **Agent Skills** e **Boost MCP Server Configuration**, e funciona para qualquer agente de código. O resultado: **181 linhas**, dentro do alvo. O que entra: a stack principal, como ativar as skills do projeto, as convenções do Laravel, verificação, estrutura e arquitetura, front-end, documentação, ser conciso nas explicações, as ferramentas do Boost (inclusive a busca na documentação do Laravel), comandos artisan, Tinker para testar models, actions e jobs, regras de PHP e de deploy, Sail, testes bons e ruins e o jeito Laravel de fazer as coisas.

Para quem não é do Laravel: o **boostify**, uma skill do curso que segue o padrão do Boost para qualquer stack de linguagem (Rust, Ruby, Python, Go) e qualquer agente. Ele detecta o projeto, configura alguns MCPs úteis (na versão do repositório, mostra o comando e você instala, passo 5) e gera um CLAUDE.md conciso, de até 200 linhas. Existe o plugin `claude-code-setup`, mas ele não faz o mesmo papel (seção 2).

A demo foi num projeto Python, o Virgo Jarvis. A skill foi copiada para o projeto, depois `/reload-skills` e `/reload-plugins`, e o boostify apareceu na lista. Ele montou as regras e perguntou "posso gravar?". Gravou um CLAUDE.md de **28 linhas**: base da stack, comandos do projeto, Transformers, uso da GPU com autocast e a orientação de consultar a documentação do Hugging Face e do PyTorch antes de usar qualquer API.



A doc a uma busca

O Boost dá ao agente uma forma de buscar na documentação do Laravel por dentro, sem deixar difícil para o modelo. O boostify faz o mesmo fora do Laravel, com o context7.

Não é inflado

CLAUDE.md não é lista de regras genéricas ("use os melhores padrões"), nem histórico de tudo que deu errado. O que não passa, sai.

> *"Target under 200 lines per CLAUDE.md file. Longer files consume more context and reduce adherence."*
>
> Documentação oficial do Claude Code · Memory (How Claude remembers your project)

2 Indo *além da aula*

ALÉM DA AULA · COMO PLUGIN, O NOME É /BOOSTIFY:BOOSTIFY

Na gravação a skill estava copiada em `.claude/skills/`, por isso era `/boostify`. Como plugin (o "pacotinho" da aula), ela ganha o prefixo: `/boostify:boostify`. Tem `disable-model-invocation: true`: escreve arquivo, então só roda quando você chama. Rode numa sessão interativa: ela mostra o bloco e só grava depois do seu "pode". No `claude -p` não tem quem responda, então ela devolve o bloco na mensagem.

Fonte: Skills, "Where skills live": plugin skill "as /plugin-name:skill-name". Teste deste material: o plugin num projeto Python (detecção, context7, proposta do bloco).

ALÉM DA AULA · AGENTS.MD OU CLAUDE.MD: A ARMADILHA DO ARQUIVO NOVO

O Boost escreve as guidelines no `CLAUDE.md` se ele já existe; senão, no `AGENTS.md`. O Claude Code lê o `AGENTS.md` só quando não há `CLAUDE.md` (nem `.claude/CLAUDE.md` ou `CLAUDE.local.md`) na pasta ou acima. Então: Boost gravou no `AGENTS.md`, depois alguém rodou `/init` ou criou um `CLAUDE.md`, e as 181 linhas somem do contexto sem aviso. A saída é a da doc: ponha `@AGENTS.md` no topo do `CLAUDE.md`, e ele importa o arquivo inteiro.

Fontes: Memory, "AGENTS.md" e "When Claude Code reads AGENTS.md" (Claude Code 2.1.277+); código do Laravel Boost 2.10.3.

ALÉM DA AULA · SAIL E WSL: CONFIRA O .MCP.JSON ANTES DO COMMIT

O `boost:install` oferece a opção do Sail quando o projeto tem `vendor/bin/sail`. Marcada, o MCP sobe com `vendor/bin/sail artisan boost:mcp` e as guidelines trazem os comandos já com o Sail. Sem Sail, dentro do WSL, o Boost 2.10.3 grava o `.mcp.json` com `wsl.exe` e caminhos absolutos da sua máquina (o PHP e o `artisan`), pensando no editor do lado do Windows. Esse arquivo não serve para o colega. Se o `/mcp` mostrar o `laravel-boost` com falha, a doc do Boost dá o comando para registrar à mão (seção 5).

Fontes: código do Boost 2.10.3 (`McpWriter`, `Sail::buildMcpCommand`); laravel.com/docs/boost, "Set Up Your Agents".

ALÉM DA AULA · ATUALIZAR SEM PERDER O QUE É SEU

Depois de subir versão de pacote: `php artisan boost:update`. A doc sugere pôr `@php artisan boost:update --ansi` no `post-update-cmd` do composer (o `laravel new` já põe, se você aceita o Boost). Guideline do time vai em `.ai/guidelines/*.md`: o Boost a inclui no bloco a cada atualização. Para cortar a peça de um pacote pouco usado: `guidelines.exclude` no `config/boost.php` (publique com `vendor:publish --tag=boost-config`), com o nome que aparece no resumo do install, como `livewire/core`. No boostify é igual: `.ai/guidelines/*.md` vira a seção "do projeto", e rodar de novo sem versão maior nova responde "bloco em dia" (o estado fica em `.ai/boostify.json`).

Fontes: laravel.com/docs/boost, "Keeping Boost Resources Updated" e "Adding Custom AI Guidelines"; Boost 2.10.3; laravel/installer.

ALÉM DA AULA · POR QUE O BLOCO É POR VERSÃO, E O CONTEXT7

O modelo escreve a API que mais viu no treino. Por isso o Boost monta as guidelines por pacote e versão maior (uma peça `core` mais a da versão instalada: Laravel 10.x a 12.x, Pest 3.x e 4.x), e o `Search Docs` consulta a doc dos pacotes que você tem. O boostify faz o mesmo: lê a versão do lockfile ou do `node_modules`, escreve só no bloco marcado e aponta para o context7.

O `npx ctx7 setup --claude` faz login por OAuth, gera a chave e pergunta o modo. **MCP**: as ferramentas do context7 entram na sessão (e o boostify já tem permissão para elas). **CLI**: instala uma skill que manda o agente usar `npx ctx7 library` e `npx ctx7 docs`; o boostify do curso também sabe usar esse modo. `--mcp` ou `--cli` escolhem direto; `-p` configura só o projeto atual. Precisa do Node.js 22.13 ou mais novo. Sem context7, o boostify roda assim mesmo, mas escreve só o que dá para afirmar pelo código, sem as regras por versão.

Fontes: laravel.com/docs/boost, "Available AI Guidelines" e "Documentation API"; README do context7 (github.com/upstash/context7), "Installation"; `npx ctx7 setup --help`.

ALÉM DA AULA · QUEM ESCREVE O QUÊ, DO LADO DA ANTHROPIC

FERRAMENTA	O QUE FAZ	ESCREVE O CLAUDE.MD?
`/init`	Analisa o código e gera um CLAUDE.md inicial; se já existe, sugere melhorias. Com `CLAUDE_CODE_NEW_INIT=1`, pergunta o que montar e mostra a proposta antes	Sim, um rascunho
`claude-code-setup`	Recomenda 1 ou 2 automações de cada tipo: MCP, skills, hooks, subagentes	Não. "It does NOT create or modify any files."
`claude-md-management`	Audita o CLAUDE.md e, com `/revise-claude-md`, propõe o que a sessão ensinou	Sim, com a sua aprovação

O bom uso do `claude-code-setup` é depois do Módulo 2: ver se ele acha um hook que você não pensou.

Fontes: README e SKILL.md dos plugins em github.com/anthropics/claude-plugins-official; Memory, "/init".

3 Faça *no seu repositório*

Dois caminhos, conforme a stack. Os passos 1 e 2 são para Laravel; os passos 3 a 8, para o resto. Os passos 9 e 10 valem para os dois.

Laravel: *Laravel Boost*

1 Instale o Boost e rode o instalador.

Criou o projeto com `laravel new` e aceitou o Boost? O pacote já está no `composer.json`: pule a primeira linha.

Terminal · projeto Laravel — laravel

```
composer require laravel/boost --dev
php artisan boost:install
# depois de subir versão de pacote: php artisan boost:update
```

Terminal · projeto Laravel com Sail — laravel

```
./vendor/bin/sail up -d
./vendor/bin/sail composer require laravel/boost --dev
./vendor/bin/sail artisan boost:install
```

Nas perguntas: marque AI Guidelines, Agent Skills e Boost MCP Server Configuration, o Claude Code entre os agentes e, se usa Sail, a opção do Sail.

2 Confira o que ele gerou.

Terminal · projeto Laravel — laravel

```
wc -l CLAUDE.md AGENTS.md
grep -l "laravel-boost-guidelines" CLAUDE.md AGENTS.md
cat .mcp.json boost.json
```

O bloco está no `AGENTS.md` e você também tem um `CLAUDE.md`? Ponha `@AGENTS.md` na primeira linha do `CLAUDE.md` (seção 2). No Claude Code, o `/mcp` precisa mostrar o `laravel-boost` conectado.

Outra stack: *boostify*

3 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

4 Instale o boostify como plugin (o pacotinho da aula).

Terminal · qualquer pasta — claude

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin marketplace update harness-lab
claude plugin install boostify@harness-lab
claude plugin list
```

A lista precisa mostrar `boostify@harness-lab` com `enabled`. Adicionou o marketplace numa aula anterior? Rodar de novo não estraga nada; para receber os plugins novos do curso, atualize:

Terminal · qualquer pasta — claude

```
claude plugin marketplace update harness-lab
```

Prefere fazer de dentro do Claude Code? O mesmo, em comandos de sessão:

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin marketplace update harness-lab
/plugin install boostify@harness-lab
/reload-plugins
```

5 Instale o context7 (a documentação por versão).

Volte para a raiz do seu projeto (`cd` no caminho dele) antes de continuar: daqui em diante, tudo roda lá.

Terminal · na raiz do seu projeto — node

```
node --version
npx ctx7 setup --claude
```

O Node precisa ser 22.13 ou mais novo. O setup abre o navegador para o login e pergunta o modo (MCP ou CLI). No modo MCP, o `/mcp` mostra o `context7`.

6 Veja o que o boostify vai ler no seu projeto.

O primeiro passo da skill é um script que só lê arquivos. Rode você mesmo para saber o que ele encontra:

Terminal · na raiz do seu projeto — shell

```
D=~/harness-lab-materiais/plugins/boostify/skills/boostify
python3 -I "$D/scripts/detecta.py" .
```

No JSON: `linguagens`, cada pacote com `versao` e `origem` (`instalada`, do lockfile ou de `node_modules`, ou `declarada`, só do manifesto), os comandos do `ci` e os `arquivos_agente` que já existem. Versão só declarada vira número aproximado no bloco.

7 Rode o boostify numa sessão interativa.

Dentro do Claude Code · na raiz do projeto — sessão

```
/reload-plugins
/boostify:boostify
```

Ele mostra o bloco com a contagem (linhas do bloco, do arquivo antes e depois) e as sugestões de hook e de skill. Só grava depois do seu "pode". Num projeto Laravel, ele para e manda usar o Boost.

8 Confira o resultado.

Terminal · na raiz do seu projeto — shell

```
wc -l CLAUDE.md
grep -n "boostify-guidelines" CLAUDE.md
cat .ai/boostify.json
```

Duas linhas no `grep` (abre e fecha), arquivo abaixo de 200. Rode `/boostify:boostify` de novo: a resposta esperada é "bloco em dia".

As duas stacks: *o que só você sabe*

9 Uma rule com `paths:` para a parte mais delicada.

Guideline de uma parte do código não precisa ocupar o CLAUDE.md. Com `paths:`, a rule só carrega quando o Claude lê ou edita um arquivo que casa com o glob. Copie o modelo e troque as linhas pelas do seu projeto:

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-3-regra
M=$A/aula-3.2-guidelines-do-seu-projeto/modelos
mkdir -p .claude/rules
cp "$M/rules/testes.md" .claude/rules/
```

10 Peça uma segunda opinião ao claude-code-setup.

Ele só recomenda, não escreve nada. Bom para conferir se ficou faltando hook, skill ou MCP. Na sessão, o `/plugin install` abre o painel do plugin: escolha o escopo e confirme.

Dentro do Claude Code — sessão

```
/plugin install claude-code-setup
/reload-plugins
> Recomende automações de Claude Code para este projeto
```

A rule que o passo 9 copiou

.claude/rules/testes.md — copiado no passo 9 · adapte

```
---
paths:
  - "tests/**"
---

# Testes

- Dados com factory, nunca seed fixo.
- Um teste por regra de negócio, com o nome da regra: "cobra frete acima de 10 kg".
- Mail, fila e HTTP externo sempre fake. (Um teste já mandou e-mail de verdade para cliente.)
```

O glob é relativo à pasta que contém o `.claude`. Outras stacks: `**/*.test.ts`, `spec/**`, `tests/**/*.py`. O Boost tem um parecido, o `.ai/rules` gravado pela ferramenta Record Rule, mas ali quem decide ler é o agente, seguindo um índice. A rule em `.claude/rules/` o próprio Claude Code carrega.

4 Arquivos que *reforçam a aula*

`skills/boostify/SKILL.md`

A filosofia do Boost em cinco passos: parar se for Laravel, detectar, consultar a doc da versão, montar o bloco com orçamento (80 linhas no bloco, 200 no arquivo) e gravar só entre as tags, depois do seu "pode".

No repositório: plugins/boostify/skills/boostify/

`references/modelo-bloco.md`

O bloco de um projeto Next.js com cada seção explicada, o equivalente no Boost e as linhas que não passam: aula do framework, estilo que o linter corrige, "nunca faça" sem exceção (hook), passo a passo longo (skill).

No repositório: plugins/boostify/skills/boostify/references/

`modelos/ai-guidelines/decisoes.md`

Onde mora o que é seu e sobrevive ao gerador: em `.ai/guidelines/`, o Boost e o boostify incluem o arquivo no bloco a cada atualização. Decisões com o porquê, uma por linha.

No repositório: modulo-3-regra/aula-3.2-guidelines-do-seu-projeto/modelos/ai-guidelines/

`modelos/rules/testes.md`

A regra de teste só importa quando ele está nos testes. Com `paths:`, fora deles ela não ocupa espaço no contexto. Ao lado, `migrations.md`, o mesmo para `database/migrations/**`.

No repositório: modulo-3-regra/aula-3.2-guidelines-do-seu-projeto/modelos/rules/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
`/boostify` não aparece	Instalado como plugin, o nome tem prefixo; ou a sessão não recarregou	`/reload-plugins` e `/boostify:boostify`
O boostify mostra o bloco e não grava	Sessão `claude -p`, sem ninguém para dizer "pode"; ou você ainda não respondeu à proposta	Sessão interativa; responda "pode" à proposta

SINTOMA	CAUSA	CORREÇÃO
As regras do Boost sumiram depois do `/init`	Com um `CLAUDE.md` na pasta, o `AGENTS.md` deixa de ser lido	`@AGENTS.md` na primeira linha do `CLAUDE.md`
`laravel-boost` com falha no `/mcp`	Containers do Sail parados, ou `.mcp.json` com `wsl.exe` e caminhos de outra máquina	`sail up -d`; ou registre à mão com o comando logo abaixo
`not found in marketplace "claude-plugins-official"`	O marketplace embutido da instalação nova é o `anthropic-plugin-directory`	Sem o `@`: `/plugin install claude-code-setup`
`npx ctx7` avisa `EBADENGINE` ou falha	Node mais velho que 22.13	Atualize o Node (nvm, mise, asdf) e rode o setup de novo
Bloco sem nenhuma regra de versão	context7 ausente, ou a doc não trouxe nada concreto (a skill não inventa regra)	Instale o context7 (passo 5), reinicie a sessão e rode de novo

●●● Terminal · projeto Laravel · MCP do Boost à mão — laravel

```
claude mcp add -s local -t stdio laravel-boost php artisan boost:mcp
```

6 Você sai *com*

- [ ] Laravel: `<laravel-boost-guidelines>` num arquivo que o Claude Code lê de fato (CLAUDE.md, ou AGENTS.md importado) e o `laravel-boost` conectado no `/mcp`.
- [ ] Outra stack: `boostify@harness-lab` em `claude plugin list` e o `<boostify-guidelines>` gravado no topo do CLAUDE.md, com `.ai/boostify.json`.
- [ ] O CLAUDE.md inteiro abaixo de 200 linhas (`wc -l`).
- [ ] Cada comando do bloco gerado rodado uma vez no terminal. Comando errado no CLAUDE.md é pior que nenhum.
- [ ] Uma rule com `paths:` em `.claude/rules/` para a parte mais delicada do código.

PROMETIDO NA AULA

- ✓ "Deixei nos materiais de apoio uma skill com um passo a passo, que eu chamei de Boostify": `plugins/boostify/` no repositório (seção 3, passos 3 a 8).
- ✓ "Vocês vão instalar isso como um pacotinho" e "o passo a passo vai estar um pouquinho diferente": marketplace `harness-lab` e `claude plugin install boostify@harness-lab` (passo 4), por isso o nome `/boostify:boostify` (seção 2).
- ✓ "Vai configurar alguns MCPs muito úteis": o context7, no lugar do `search-docs` do Boost, e o MCP do próprio Boost no Laravel (passo 1). Na aula ele configura; no repositório, ele mostra o comando e você instala (passo 5).
- ✓ "Um CLAUDE.md de até 200 linhas, conciso e bem estruturado": o orçamento está no `SKILL.md` (80 no bloco, 200 no arquivo).
- ✓ O `sail artisan boost:install` da gravação: passo 1, com e sem Sail.
- ✓ "Tem um plugin Claude Code Setup": o que ele faz de verdade (seção 2) e como usar (passo 10).

7 Glossário

Guideline	Orientação para o agente: padrão, convenção, versão. Pode ter exceção. O que não pode é hook.
Bloco marcado	O trecho entre tags que o gerador troca a cada atualização. O resto do arquivo é do time.
.ai/guidelines	Pasta da guideline própria, que o gerador inclui no bloco a cada vez.
context7	MCP ou CLI que entrega a documentação de uma biblioteca na versão pedida.
Rule com paths	Arquivo em `.claude/rules/` que só carrega quando o Claude mexe num arquivo que casa.
AGENTS.md	Arquivo de instrução lido por vários agentes. O Claude Code lê quando não há CLAUDE.md.

8 Para *ir além*

- Laravel Boost: instalação, MCP, guidelines por versão, `.ai/guidelines`, skills customizadas e `boost:update`.
- Memory: a meta de 200 linhas, AGENTS.md, imports com `@` e rules com `paths:`.
- Discover and install plugins: marketplace, escopos e `/reload-plugins`.
- context7: modos MCP e CLI, ID de biblioteca e versão no pedido.
- claude-plugins-official: o `claude-code-setup` e o `claude-md-management` por dentro.

Próxima: *Aula 3.3 · Skills: conhecimento sob demanda*

E o passo a passo de 30 linhas que você tirou do CLAUDE.md? Ele não some: vira skill e só entra quando precisa. O boostify é uma skill, e a 3.3 explica por quê.